

Circuit Tracing in Vision–Language Models: Understanding the Internal Mechanisms of Multimodal Thinking

Jingcheng Yang^{1*} Tianhu Xiong^{1*} Shengyi Qian^{2†} Klara Nahrstedt¹ Mingyuan Wu¹

¹University of Illinois Urbana-Champaign ²Independent Researcher

{jy95, mw34, klara}@illinois.edu

Abstract

Vision–language models (VLMs) are powerful but remain opaque black boxes. We introduce the first framework for transparent circuit tracing in VLMs to systematically analyze multimodal reasoning. By utilizing transcoders, attribution graphs, and attention-based methods, we uncover how VLMs hierarchically integrate visual and semantic concepts. We reveal that distinct visual feature circuits can handle mathematical reasoning and support cross-modal associations. Validated through feature steering and circuit patching, our framework proves these circuits are causal and controllable, laying the groundwork for more explainable and reliable VLMs. Our code and models are available at <https://github.com/UIUC-MONET/vlm-circuit-tracing>.

1. Introduction

The rapid advancement of vision–language models (VLMs) has fundamentally transformed how machines understand and reason about multimodal information. Contemporary VLMs such as CLIP [29], Flamingo [1], LLaVA [20], and GPT4-o [35] demonstrate remarkable capabilities in tasks ranging from visual question answering and image captioning to complex visual reasoning and embodied AI applications. These models seamlessly integrate visual perception with linguistic understanding, enabling machines to answer questions about images, generate detailed descriptions of visual scenes, and even perform multi-step reasoning that requires coordinating information across modalities. Despite these impressive empirical successes, a critical question remains largely unanswered: *How do these models actually work internally?*

Understanding how VLMs work is essential for building trustworthy, controllable AI. Although they are used in high-stakes areas like medical imaging, autonomous driving, and content moderation, their decision-making remains

<start_of_image> This is the image of a sea^{otter}

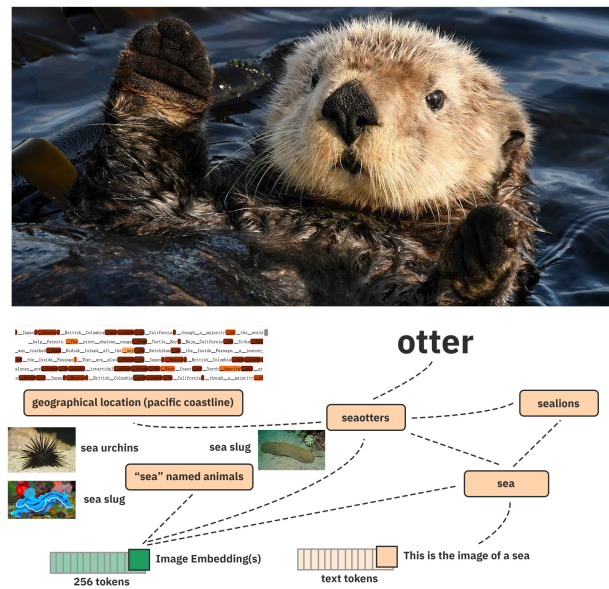


Figure 1. Given an image and a prompt, how can we extract a circuit, as an internal computation graph, of open-source vision language models such as Gemma3-4B [34] from Google. We introduce the first framework for successful circuit tracing in VLMs, enabling analysis of the internal circuits and association of concepts in underlying multimodal reasoning.

opaque. This lack of interpretability makes it hard to diagnose errors, mitigate biases, and ensure alignment with human values. It also limits scientific insight—reverse-engineering their mechanisms can reveal how vision and language interact and guide the design of more capable, efficient architectures.

Recent work in interpretability has begun revealing the internal algorithms used by language models through attention visualization [33], probing, and circuit discovery [3, 15]. Yet these methods focus almost entirely on text-only models. VLMs introduce deeper challenges: they must integrate two modalities with different statistics and seman-

* Jingcheng and Tianhu contributed equally.

† This work was conducted independently and is not related to the author’s employment at Meta FAIR.

tics while discovering meaningful visual–linguistic correspondences. How VLMs bind visual features to tokens, implement cross-modal reasoning, or coordinate visual and linguistic attention remains largely unknown—posing a more complex frontier than interpretability work in single-modality text models or early visual interpretability research [26, 38].

This work introduces the first framework for successful circuit tracing in VLMs, enabling systematic analysis of the internal computational mechanisms underlying multimodal reasoning. Our approach builds on recent advances in interpretability for language models. Specifically, we leverage transcoders [9] to decompose neural representations into interpretable features, and combine these with attribution-based circuit discovery methods [3, 15, 30] to map causal relationships between features. We are the first to extend these techniques to the multimodal setting. In doing so, we address the unique challenges posed by vision–language integration and develop new methods to trace information flow from visual inputs through the model’s reasoning process to final outputs.

Extending circuit tracing to the multimodal domain is far more challenging than we thought, but brings richer insights and opportunities in VLMs than we anticipate.

Make Circuit Tracing Happen in VLMs. We build three key components in VLMs:

1. *Transcoders in VLM*: We insert and train per-layer transcoders into VLMs to decompose multimodal, polysemantic representations into interpretable, monosemantic features. (Sec. 3.1)
2. *Attribution Graph*: We trace attribution graphs that incorporate image-embedding residuals to reveal causal relations between features in the resulting multimodal computational graph. (Sec. 3.2)
3. *Circuit Discovery of Visual and Text Tokens*: We propose attention analysis to interpret unnamed multimodal features, allowing for fully interpretable multimodal circuit discovery. (Sec. 3.3)

Representative Insights from Multimodal Circuits. These circuits are not mere post-hoc correlations but represent genuine causal mechanisms which help interpret VLM internal computation. In Section 5 we include:

1. *Hierarchical Integration of Visual and Semantic Concepts*: Features that simultaneously encode both semantic and visual concepts emerge only in higher layers of the network.
2. *Highly Interpretable Case Studies*: for Visual Math reasoning, Six Finger Hallucination, Association between Mars and Space Shuttle.
3. *Distinct Visual Latent Space in Language Model*: We find that visually similar features cluster and co-activate, indicating that the language-model component of the VLM preserves a distinctly visual representation space.

Intervention with Multimodal Circuits. Intervention means changing the activation of features in the circuits, enabling more opportunities:

1. *Steering*: We modify the activation of certain features to observe the impact it has on the final output. (Sec. 3.5)
2. *Circuit Patching*: We transfer entire patches of one circuit to another circuit of similar structure and function, to see whether the transplanted circuit performs the same. (Sec. 4.5)

In summary, our contributions are threefold: we establish the first circuit tracing framework for VLMs, uncover key insights into multimodal reasoning with circuits, and demonstrate through intervention experiments the potential for circuit-based model manipulation and control.

2. Related Work

Mechanistic interpretability in LLMs aims to causally reverse-engineer how model components implement behavior, going beyond earlier correlational interpretability methods such as probing and attention analysis [5, 16, 36]. In language models, this literature includes circuit-level analyses of specific behaviors (e.g., induction heads, IOI, and factual recall/localization) [12, 22, 23, 27, 37], intervention-based localization methods such as activation patching and path patching [14, 40], and feature-level approaches motivated by superposition, where sparse autoencoders are used to recover more interpretable features than individual neurons [4, 11, 13]. The field has also become more scalable through partially automated circuit discovery (ACDC) and shared tooling platforms such as Neuronpedia [6, 18].

Sparse autoencoders and transcoders. Neural representations are often polysemantic, responding to multiple unrelated concepts [32]. Sparse autoencoders (SAEs) [10] mitigate this by learning sparse, overcomplete decompositions that yield more interpretable, often monosemantic features [8]. Transcoders [9] extend this idea by replacing MLP layers with SAEs trained to match the MLP’s outputs, preserving the model’s computation while exposing feature-level structure. This enables end-to-end attribution and circuit tracing. While transcoders have been used to analyze language models [9], their application to VLMs remains unexplored.

Vision–language model interpretability. While interpretability methods have advanced significantly for language models, understanding vision–language models remains challenging. Most existing work on VLM interpretability focuses on high-level analysis: attention visualization between image and text tokens [33], probing for learned visual concepts [26], and analyzing cross-modal alignment [29]. More recently, several works have attempted to localize visual knowledge in VLMs [25] or understand how visual information is processed through the

network [17]. However, these methods typically analyze individual components in isolation or rely on correlation-based analysis rather than causal circuit discovery. Our work is the first to apply circuit tracing methodology to VLMs, enabling systematic identification of the computational mechanisms underlying multimodal reasoning.

3. Circuit Tracing in VLMs

In this section, we present our framework for circuit tracing in vision–language models. We first describe how we train transcoders to decompose MLP activations into sparse, interpretable features (Section 3.1). We then explain how attribution graphs are computed to capture causal relationships between features across layers (Section 3.2). Next, we detail our approach to analyzing and interpreting learned features through activation patterns and attention visualization (Section 3.3). We describe our circuit discovery algorithm that identifies minimal computational subgraphs responsible for specific capabilities (Section 3.4). Finally, we present intervention and steering methods to causally validate discovered circuits (Section 3.5).

3.1. Transcoders

Sparse Autoencoders (SAEs) [8] provide a sparse, interpretable basis for transformer activations, but they do not cleanly expose causal relations between features, limiting circuit discovery. Whereas SAEs are trained directly to reconstruct a transformer’s activations, transcoders [9] replace a transformer’s MLP sublayer with a sparse autoencoder, maintaining computational equivalence while enabling feature-level analysis.

For each MLP layer in the vision–language model, we train a transcoder consisting of an encoder and decoder. The encoder maps the MLP input $x \in \mathbb{R}^{d_{\text{model}}}$ to $d_{\text{feat}} \gg d_{\text{model}}$ learned latent features via:

$$z(x) = \text{ReLU}(W_{\text{enc}}x + b_{\text{enc}}), \quad (1)$$

where $W_{\text{enc}} \in \mathbb{R}^{d_{\text{feat}} \times d_{\text{model}}}$ and $b_{\text{enc}} \in \mathbb{R}^{d_{\text{feat}}}$ are learned parameters. The decoder reconstructs an approximation to the original MLP output:

$$\text{TC}(x) = W_{\text{dec}}z(x) + b_{\text{dec}}, \quad (2)$$

where $W_{\text{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{feat}}}$ and $b_{\text{dec}} \in \mathbb{R}^{d_{\text{model}}}$.

Unlike the original transcoder implementation [9], which uses an ℓ_1 penalty to encourage sparsity, we follow EleutherAI [10] and enforce sparsity directly via $\text{TopK}(z(x), k)$, retaining only the k largest activations. This removes the need for a sparsity coefficient and yields more stable training with consistent sparse features. We evaluate reconstruction quality using the *Fraction of Variance Unexplained* (FVU):

$$\text{FVU} = \frac{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2} = \frac{\text{MSE}}{\text{Var}(y)}. \quad (3)$$

Where y is $\text{MLP}(x)$ (the original MLP output) and $\hat{y}$ is $\text{TC}(x)$. Training minimizes only the reconstruction error, while sparsity is controlled solely by the choice of k (rather than through a tunable penalty). Each transcoder feature is defined by a paired encoder column and decoder row, contributing additively to the output. The expanded latent space ($d_{\text{feat}} = N_{\text{latents}} \cdot d_{\text{model}} \cdot N_{\text{layers}}$) enables polysemantic MLP representations to be factorized into sparse, approximately monosemantic, and thus interpretable features.

While transcoders have been extended to model cross-layer structure [3], we use the per-layer formulation as originally introduced by Dunefsky et al. [9]. After training a transcoder for each MLP block, we construct a replacement model by substituting each MLP with its corresponding transcoder, yielding a network expressed entirely in sparse, learned latent features. Because transcoders only approximate the original MLPs, we explicitly track the reconstruction residual

$$e(x) = \text{MLP}(x) - \text{TC}(x), \quad (4)$$

computed from cached MLP outputs and included as a separate error node in the circuit graph. This accounts for approximation error without altering the forward pass of the replacement model.

3.2. Attribution Graphs

To trace circuits, we compute an attribution graph as introduced by Lindsey et al. [19] and Ameisen et al. [3], adapted for per-layer transcoders by Hanna et al. [15], that linearly decomposes how features contribute to activations in upper layers and, ultimately, to the output logits on a fixed prompt. Because each transcoder replaces the MLP with a sparse linear readout from features, and all nonlinearities (ReLU, attention patterns, and normalization factors) are frozen at their values on the given prompt, the model becomes locally linear around that input.

Each node in the graph corresponds to a token embedding, an active transcoder feature at a particular (layer, position) pair, or an output logit. For a source node s and target node t , the attribution is defined as:

$$A_{s \rightarrow t} = a_s w_{s \rightarrow t}, \quad (5)$$

where a_s is the activation magnitude of source feature s and $w_{s \rightarrow t}$ is the *virtual weight*—the local derivative of t ’s pre-activation with respect to s ’s activation. For per-layer transcoders, this weight factors into the decoder of s , the frozen transformer Jacobian, and the encoder of t :

$$w_{s \rightarrow t} = f_{\text{dec}}^{(s)\top} J_{(s) \rightarrow (t)}^{\nabla} f_{\text{enc}}^{(t)}, \quad (6)$$

where $f_{\text{dec}}^{(s)} \in \mathbb{R}^{d_{\text{model}}}$ is the decoder vector for feature s , $f_{\text{enc}}^{(t)} \in \mathbb{R}^{d_{\text{model}}}$ is the encoder vector for feature t , and $J_{(s) \rightarrow (t)}^{\nabla} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ is the residual-stream Jacobian

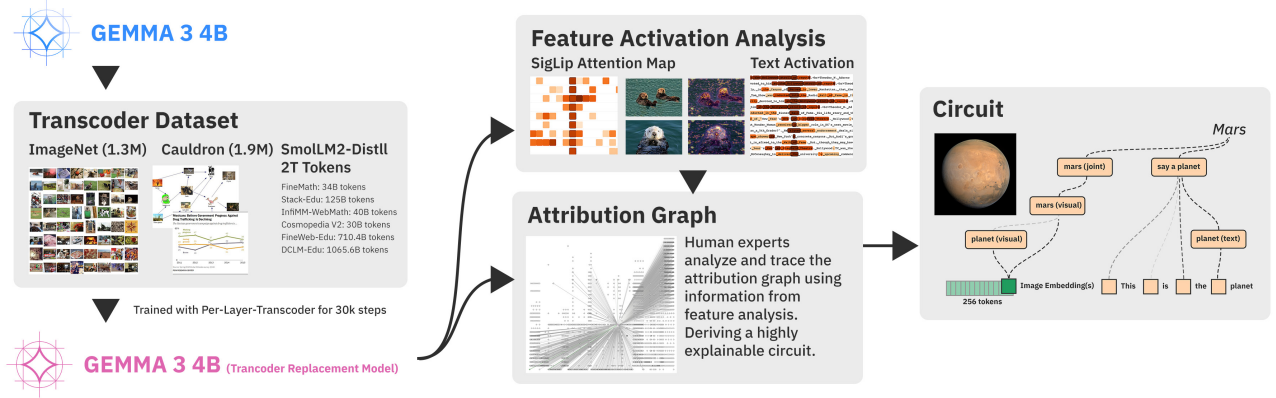


Figure 2. A summary of our method, we first train transcoders for Gemma-3-4b-it on our curated dataset, yielding a replacement model with monosemantic features. Then, we utilize feature activation analysis to obtain information that would provide interpretable information about the features. Then, we generate an attribution graph of a given prompt, and utilize human experts to derive the final circuit.

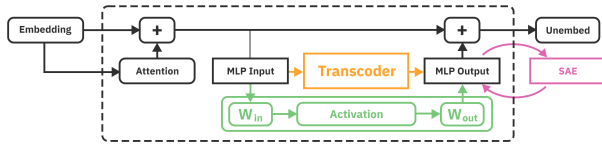


Figure 3. Transcoder vs SAE; SAEs learn to reconstruct model activations, whereas transcoders imitate MLP sublayers’ input-output behavior.

of the original transformer with stop-gradients on all non-linearities (LayerNorm, attention softmax, and ReLU).

Since the model is linearized, the pre-activation of each node is exactly the sum of its incoming attributions:

$$h_t = \sum_{s \in \text{pred}(t)} A_{s \rightarrow t}, \quad (7)$$

yielding a complete additive explanation of how earlier representations influence later ones. We construct the full directed attribution graph $G = (V, E)$ for each prompt, where nodes V include all token embeddings, active features, and output logits, and edges E have weights $A_{s \rightarrow t}$. We prune edges with negligible attribution ($|A_{s \rightarrow t}| < \epsilon$) to produce a sparse, interpretable graph.

3.3. Feature Interpretation and Attention Analysis

Feature activation analysis. To understand what each transcoder feature represents, we analyze its activation patterns across a diverse dataset of vision-language inputs. For each feature f_i , we collect the top- k activating examples—image-text pairs (I, T) that produce the highest activation z_i —and examine commonalities. We compute activation statistics including the feature’s activation frequency (fraction of inputs where $z_i > 0$), mean activation magnitude, and position distribution (which tokens or image patches most frequently activate the feature).

Vision encoder attention maps. While feature activations on text tokens are directly interpretable, activations on image tokens remain opaque. To visualize which image regions the SigLIP vision encoder attends to, we compute attention-rollout maps for each of the visual embeddings passed to the language model.

In the SigLIP vision-encoder for Gemma 3, it processes a resized 896×896 image and reduces the representation to a fixed set of $N_v = 256$ visual tokens that are then input to the language model.

We compute attention-rollout maps over the last K self-attention layers of the vision tower. Let the encoder output T tokens in total (including possibly a class token), of which N_v are the visual patch tokens entering the language model. For each of the last K layers ℓ , we select the fraction q of attention heads with lowest mean entropy (i.e., the most focused heads), average their $(T \times T)$ attention matrices to obtain $\tilde{A}^{(\ell)}$, add an identity residual, and row-normalize to form a row-stochastic matrix $\hat{A}^{(\ell)}$. The rollout is

$$R = \tilde{A}^{(L-K+1)} \tilde{A}^{(L-K+2)} \dots \tilde{A}^{(L)} \in \mathbb{R}^{T \times T}. \quad (8)$$

We then extract the $N_v \times N_v$ sub-matrix R_{vis} corresponding to the visual tokens and interpret each row as an attention distribution over the 256 visual token positions. To provide spatial localization, we recover the original spatial grid of image patches (e.g., $g_h \times g_w$) from the encoder, pool tokens within non-overlapping blocks of size $b \times b$, reshape into a $g_h \times g_w$ grid, upsample to the input image resolution, and normalize each map. The resulting grayscale heat-maps indicate which image regions are most attended to in the vision tower. We chose not to add additional refinements, denoising or post-processing so as to present the raw information for human experts to analyze.

3.4. Circuit Discovery

A *circuit* is an abstract representation of the computational graph that explains a model’s output logits for a given input. Instead of tracking every feature activation, through explaining the feature representations as described in Section 3.3, we can group features of similar functions into shared nodes, resulting in a simplified graph that explains the model’s behavior.

This mechanistic structure highlights the key computational pathways and interactions that drive the model’s behavior, allowing us to steer and intervene in the model’s activations towards predictable directions.

While there are now methods for large-scale, partially automated circuit discovery [7], a human-annotated circuit remains the most accurate and interpretable representation of a model’s mechanism. In this work we therefore use human experts to discover and annotate circuits.

3.5. Intervention and Steering

To study how individual transcoder features influence model behavior, we directly modify feature activations during the forward pass and observe the resulting changes in the model’s output.

Feature representation. At layer ℓ and position t , the transcoder produces feature activations $z_{\ell,t,i} \in \mathbb{R}$ and each feature has an associated decoder vector $d_{\ell,i} \in \mathbb{R}^{d_{\text{model}}}$ that determines how changes to that feature are written back into the residual stream.

Interventions. An intervention specifies a target value $v_{\ell,t,i}$ for a feature. During the forward pass we compute

$$\Delta z_{\ell,t,i} = v_{\ell,t,i} - z_{\ell,t,i}(x), \quad (9)$$

and apply the corresponding residual update

$$h_{\ell,t} \leftarrow h_{\ell,t} + \Delta z_{\ell,t,i} d_{\ell,i}. \quad (10)$$

This simply adjusts the model’s internal representation as if the feature had taken the desired value.

Circuit patching refers to directly overwriting selected internal activations during the forward pass, or transplanting entire sub-circuits from another circuit, and observing how the model’s output changes. In our case, we patch transcoder features by setting them to chosen target values (e.g., zero for ablation or a positive constant for amplification) at specific layers and positions. We observe whether transferring a patch from Circuit A onto B would recreate similar behaviors of A for B.

All experiments are performed on the transcoder-replaced model, and we qualitatively inspect the differences in predictions to understand the behavioral role of the intervened features.

4. Experiments

Model. We conduct our circuit tracing experiments on Gemma-3-4B-it [34], a state-of-the-art open-source vision–language model. Gemma-3-4B-it processes images using a SigLIP [39] vision encoder, which produces patch tokens that are concatenated with text tokens and fed into a transformer decoder. This architecture enables joint visual–linguistic processing and makes the model suitable for multimodal reasoning.

Architecture. The model consists of a SigLIP encoder with patch size 14, applied to 896×896 input images. This produces a grid of 64×64 patches, yielding 4096 initial patch tokens [34]. These tokens are then pooled into 256 soft image tokens before being appended to the text sequence for decoding. Gemma-3 models employ bidirectional attention over image tokens, allowing the model to attend to the full image context throughout processing.

The decoder is a 34-layer transformer with $d_{\text{model}} = 2560$, 34 attention heads, and MLP dimension $d_{\text{ff}} = 10240$.

4.1. Training Transcoders

Name	Content	Size
SmoLIM2-135M-10B [2]	2048 text tokens	144,000
ImageNet [31]	1 image; Caption	144,000
Cauldron [17]	1 image; QA Text	72,000

Table 1. Datasets used for Transcoder Training.

Datasets. We train transcoders on a diverse corpus of vision–language examples to ensure broad coverage of multimodal concepts and reasoning patterns. Our training dataset consists of a base set of pure text dataset for broad coverage of various domains [2], alongside ImageNet [31] for rich visual features and Cauldron [17] consisting of QA and visual reasoning tasks. For the cauldron split, we sampled from its 50 subsets evenly to ensure even coverage.

Implementation details. For transcoder training, we implemented our framework with support for Gemma-3 on top of Sparsify [10].

Training configurations. For every MLP layers in Gemma-3-4B-it, we train a separate transcoder with $d_{\text{feat}} = [N_{\text{latents}} \cdot d_{\text{model}} \cdot 34]$ features. N_{latents} is the number of features in the hidden layer of a single transcoder block, which we then multiply by 34, the number of decoder layers in Gemma-3-4B. During the forward pass of the transformers, we only activate the top k latents as described in Section 3.1, for our experiment we set k to 48.

Transcoder training uses the AdamW [21] optimizer with a learning rate $2 \times 10^{-4} \times \sqrt{\frac{2^{14}}{N_{\text{latents}} \times d_{\text{model}}}}$, with $N_{\text{latents}} = 64$, $d_{\text{model}} = 2560$. We also apply a linear warmup over

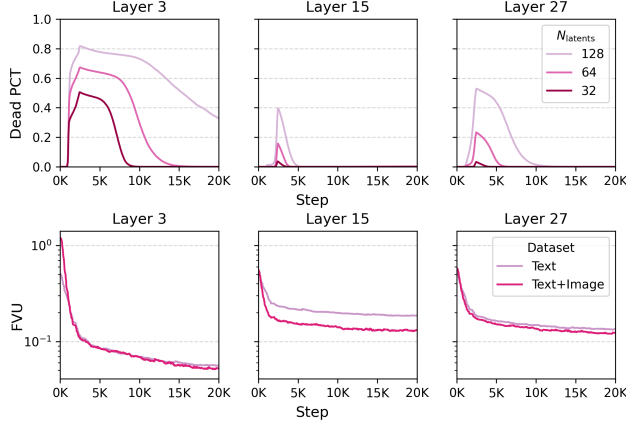


Figure 4. **Top:** Percentage of dead latents (Dead PCT) across layers for different values of N_{latents} . **Bottom:** Fraction of variance unexplained (FVU) when training transcoders on a text-only dataset compared to our multimodal split.

the first $N_{\text{warmup}} = 1,000$ steps. Training uses a batch size of 12 and runs for 30,000 steps on 8 H100 GPUs for approximately 60 hours.

Expansion Factor. We trained transcoders with different expansion factor $N_{\text{latents}} \in \{32, 64, 128\}$. As shown in the top panel of Figure 4, the choice of N_{latents} substantially affects the proportion of dead latents. We define a latent as *dead* if it fails to activate above a minimal threshold for an extended portion of training. A high fraction of dead latents indicates that the model is not utilizing the full capacity of the latent space, suggesting limited diversity or overlap of concepts represented within the corresponding features. The curves also illustrate how this behavior varies across layers: early layers (e.g., Layer 3) exhibit substantially higher dead-latent ratios, whereas mid-network layers (e.g., Layer 15) show far denser activation patterns.

FVU and multimodality. The bottom panel of Figure 4 compares the FVU obtained when training transcoders on our text-only split (SmoLIM2-135M-10B) versus our full multimodal (text+image) dataset. Although FVU is computed in-domain and does not directly reflect out-of-domain explainability, transcoders trained with multimodal supervision consistently achieve lower FVU across all layers. This suggests that visual features provide additional constraints that make the underlying representations more explainable. The gap is largest in the middle layers (e.g. Layer 15), consistent with our broader findings (Sec. 5) that this is where visual information is integrated into unified semantic representations; thus, explaining these circuits requires both modalities. At higher layers, the representation space becomes more uniform, and the FVU gap correspondingly decreases.

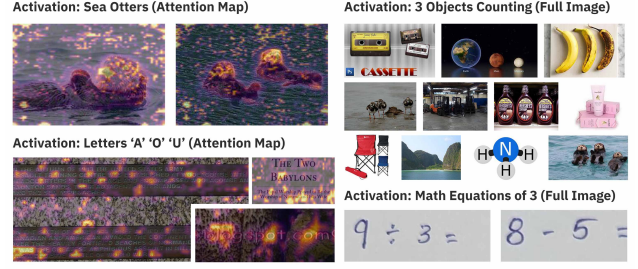


Figure 5. Feature activations on images with a mix of feature annotations resulting from full images (right) and feature analysis resulting from noisy attention maps (left).

4.2. Computing Attribution Graphs

Implementation details. We computed attribution graphs using the circuit tracer library by Hanna et al. [15], where we extended support for Gemma-3 VLMs. Images in Gemma-3 are not masked [34], achieving full attention and allowing the model to see every part of the image in a bidirectional manner. Our attribution graphs are adapted to reflect this. To properly monitor residual streams and activation of individual transformer blocks, we utilize the TransformerLens library by Nanda and Bloom [24], with extended support for VLMs [25].

Attribution. For attribution graph computation, we continuously add nodes and edges to our attribution graph until it reaches a cumulative influence thresholds 0.8 and 0.98 respectively.

Circuit discovery uses the top- m features ranked by partial influence, where m controls the maximum number of feature nodes ($m = 7500$), and we select up to 10 logit nodes whose cumulative probability mass is at least 0.95. All attribution experiments run in bfloat16 precision to reduce memory usage. Computing an attribution graph for a simple image-text QA task takes 20 minutes on a single H100 GPU.

4.3. Feature Analysis

Datasets. Our dataset for feature analysis is a subset of our transcoder training dataset. Including image-text pairs sampled from ImageNet (18,000 images) and Cauldron (10,000 images) [17, 31] as well as a text-only split of $10,000 \times 2048$ tokens [2]. For circuit discovery and validation, we use a task-specific dataset consisting of a small, curated sample of 20-100 internet images that are semantically related to the image we want to trace. Our results demonstrate this to be a highly effective and efficient method at improving a graph’s interpretability.

Implementation Details. For computing activations, we base our method upon SAE Dashboard [30] for LLM feature activation analysis. We insert hookpoints at every MLP sublayer’s input and output to record activation patterns us-

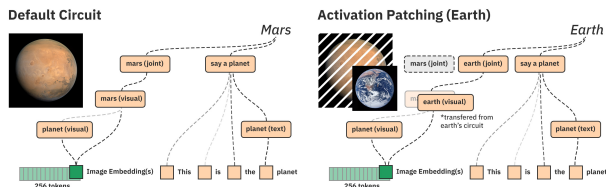


Figure 6. We conduct circuit tracing on the same prompt *This is the planet* with images of mars and earth. We then conduct activation patching by suppressing the mid-layer visual features of mars in the mars circuit, and set the activation of earth visual features discovered in the earth circuit in the mars circuit

ing TransformersLens extended with VLM support [24, 25]. For activations upon image tokens, we store the index of the image tokens and the image in order to retrieve its attention map during circuit discovery.

Ad Hoc Feature Analysis. Whereas circuit tracing tools [15, 18] for LLMs precompute all feature activations, we selectively compute only the features in a given attribution graph, which usually contains under 2000 features given our pruning parameters, significantly reducing the compute and storage costs of caching activation data.

While our curated dataset allows for general interpretability of various features in the attribution graph, it may not be sufficient. In our attribution graph analyzing the image of a sea otter, further calculating the feature activations on a curated, small dataset of 30 sea otter images significantly increased the feature’s interpretability and allows us to be more certain of a feature’s function.

Computing feature activation on a single attribution graph with approx. 1000 features takes 20 H100 GPU hours on our dataset, with a negligible increase when injected with a curated dataset.

Attention Maps. We extract the custom SigLip encoder from Gemma-3-4B-it, and use that to compute attention maps for images using our method described in Section 3.3. Computing attention maps is fast, and we pre-compute attention maps for all the 28,000 images, which takes about 2 hours on a single H100 GPU and results in approx. 2TB of single channel masks that are recombined with the base image when queried.

4.4. Finding Circuits

To obtain the most accurate circuits for validation, we use human experts (Sec. 3.4) to compute the subgraph. We manually identify features that exhibit a similar function (e.g. represent similar concepts, has similar effects on model behavior), and group them into distinct nodes. The attribution between nodes is the sum of the attribution exerted by the features in each node. The resulting graph of nodes is generally an explainable, simplified circuit.

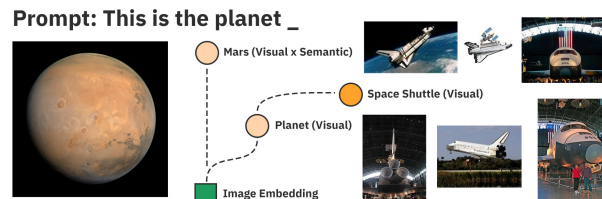


Figure 7. Tracing a purely visual attribution path from an image of Mars reveals internal visual associations (e.g., “space shuttle”) even without supporting cues.

4.5. Intervention Experiments

We use a combination of steering and activation patching (Sec. 3.5) to validate our circuits. Detailed experiments are available in the appendix. One example of activation patching is shown in figure 6, where we suppressed features representing the visual concept of Mars, and instead activated features representing the visual concept of Earth identified in the earth circuit, and the subsequent feature activations and final output all changed to earth-related concepts.

5. Empirical Findings

Our circuit tracing framework reveals several core principles underlying how VLMs such as Gemma3 integrate and manipulate multimodal information. The results validate our methodology and offer a clearer picture of the computational structure supporting vision-language reasoning.

Hierarchical Formation of Multimodal Representations.

We observe a progressive integration of visual and semantic information as network depth increases. Only in higher layers (emerging around Layer 20) do features jointly encode both visual and semantic concepts. Earlier layers remain largely modality-specific, supporting the progressive binding hypothesis in which cross-modal associations are gradually assembled across depth.

Granularity and Monosemanticity Across Layers.

Feature representations become increasingly abstract with depth. Early layers exhibit highly localized, fine-grained visual patterns—down to digits or textures—while later layers form object- and concept-level features, paralleling trends in vision models [25, 38] but now aligned with semantics.

Visual Circuits in Simple Mathematical Reasoning.

For image-based arithmetic (e.g., $1 + 2$ rendered visually), the model appears to compute partially within visual space. Intermediate layers contain visual features corresponding to the resulting numeral (e.g., “3”), activating across contexts. We also identify visual encodings of digit ranges and modular arithmetic patterns, echoing textual findings from Lindsey et al. [19]. These results suggest that simple arithmetic over images can rely on visual circuits rather than purely semantic computation.

Understanding Hallucination: The Six-Finger Problem.

Our tracing suggests that hallucinations—such as the six-finger case—arise from an interaction between perceptual bias and internal circuit dynamics rather than from a single failure mode. The vision encoder appears to produce embeddings that heavily emphasize generic “hand” semantics, while the model’s internal circuits further amplify these features. As a result, visual features corresponding to the digit “6” are suppressed toward the level of unrelated numbers, whereas hand-related features strongly activate the “five” circuit. Although the model possesses circuits capable of visual counting (Fig. 5), these can be overshadowed by more dominant semantic and perceptual signals, showing how both encoder-level weighting and feature competition jointly contribute to hallucination.

Parallel Visual and Semantic Pathways with Late Convergence. Gemma3 maintains distinct visual and semantic representation streams deep into the network. We identify visually grounded associative features—such as “space shuttle” activations triggered by an image of Mars (Fig. 7)—reflecting internal visual associations independent of semantics. High-level layers also preserve visual similarity (e.g., consistent activations for sea otters, seals, and beavers) even when semantic categories diverge. These streams ultimately merge in the final layers, where they align into a unified multimodal representation supporting coherent reasoning.

6. Conclusion

This work introduces the first circuit-tracing framework for vision–language models, revealing the mechanisms underlying multimodal reasoning. Using transcoders to extract interpretable features and attribution graphs to trace causal structure, we isolate sparse circuits for tasks such as object recognition, counting, QA, and captioning. Intervention experiments confirm these circuits are causally meaningful, enabling targeted impairment and controllable steering. Beyond advancing scientific understanding, this framework provides practical tools for debugging, mitigating failures, and guiding more interpretable VLM design—supporting the development of transparent, controllable, and aligned AI systems.

7. Limitations and Future Work.

Despite these contributions, many important limitations remain. First, the vision-encoder attention maps used for interpreting visual features can be difficult to read: they sometimes fail to localize relevant regions or provide meaningful contextual cues, limiting their utility for feature annotation. In addition, for tasks requiring additional processing of visual inputs (e.g., math operations), it is difficult to distinguish between features that mediate the computation and features that represent its output.

Second, our use of per-layer transcoders cannot capture cross-layer superposition [3], which could be a more significant drawback for VLMs due to the high feature density of image embeddings; we frequently observe many near-duplicate visual features firing in attribution graphs, suggesting the need for cross-layer or more adaptive transcoder designs. In addition, our work does not investigate in detail regarding specific choices of the transcoder configuration, under the assumption that they are largely the same. Research into specific SAE methods (e.g. JumpRelu, BatchTopK, etc) and varying configurations for optimal transcoder training for VLMs could prove to be useful.

Thirdly, by largely mirroring the circuit-tracing method for LLMs [3], the computational cost for understanding and explaining multimodal features have risen significantly. While we accommodated for this limitation by computing feature activations ad-hoc, we believe methods for comprehensive or automated interpretation of features could greatly reduce the complexity of our current process. Current methods for automatic feature interpretation [28] are computationally too expensive, and we call for future works to improve this process.

In addition, our current analysis is limited to one specific model. We have reason to suspect some complexities in circuit-tracing might be complicated by Gemma3’s SigLip and bidirectional-attention mechanism. Extending this work to accommodate a wider collection of VLMs could greatly solidify findings and conclusions in our current work.

Finally, circuit discovery currently requires substantial human effort, making it difficult to introduce quantitative evaluation or apply our method directly to model fine-tuning or improvement. Automating feature labeling, simplifying attribution graphs, or developing cross-layer transcoders could help scale circuit tracing to larger models and enable more systematic evaluation.

8. Acknowledgements

This research used the Delta advanced computing and data resource, which is supported by the National Science Foundation (award OAC-2005572) and the State of Illinois. Delta is a joint effort of the University of Illinois Urbana-Champaign and its National Center for Supercomputing Applications. This work was supported by the National Science Foundation under awards CNS-2106592 and CCF-2217144. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob L Menick, Sebastian Borgeaud, Andy Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Bińkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karén Simonyan. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, pages 23716–23736. Curran Associates, Inc., 2022. 1
- [2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgrén, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smolm2: When smol goes big – data-centric training of a small language model, 2025. 5, 6
- [3] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. 1, 2, 3, 8
- [4] Trenton Bricken et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits*, 2023. 2
- [5] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does bert look at? an analysis of bert’s attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP*, 2019. 2
- [6] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adria Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. *arXiv preprint arXiv:2304.14997*, 2023. 2
- [7] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adria Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023. 5
- [8] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. 2, 3
- [9] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable llm feature circuits, 2024. 2, 3
- [10] EleutherAI. Sparsify: Transformers with saes and transcoders, 2025. GitHub repository, accessed 13 November 2025. 2, 3, 5
- [11] Nelson Elhage, Tristan Hume, Catherine Olsson, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022. 2
- [12] Nelson Elhage et al. A mathematical framework for transformer circuits. *Transformer Circuits*, 2021. 2
- [13] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *EMNLP*, 2021. 2
- [14] Nicholas Goldowsky-Dill, Chris MacLeod, Lucas Sato, and Aryaman Arora. Localizing model behavior with path patching. *arXiv preprint arXiv:2304.05969*, 2023. 2
- [15] Michael Hanna, Mateusz Piotrowski, Jack Lindsey, and Emmanuel Ameisen. circuit-tracer. <https://github.com/safety-research/circuit-tracer>, 2025. The first two authors contributed equally and are listed alphabetically. 1, 2, 3, 6, 7
- [16] Sarthak Jain and Byron C. Wallace. Attention is not explanation. In *Proceedings of NAACL-HLT*, 2019. 2
- [17] Hugo Laurençon, Léo Tronchon, Matthieu Cord, and Victor Sanh. What matters when building vision-language models?, 2024. 3, 5, 6
- [18] Johnny Lin. Neuronpedia: Interactive reference and tooling for analyzing neural networks, 2023. Software available from neuronpedia.org. 2, 7
- [19] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025. 3, 7, 5
- [20] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, pages 34892–34916. Curran Associates, Inc., 2023. 1
- [21] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. 5
- [22] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *arXiv preprint arXiv:2202.05262*, 2022. 2
- [23] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022. 2
- [24] Neel Nanda and Joseph Bloom. Transformerlens. <https://github.com/TransformerLensOrg/TransformerLens>, 2022. 6, 7
- [25] Yaniv Nikankin, Dana Arad, Yossi Gandelman, and Yonatan Belinkov. Same task, different circuits: Disentangling modality-specific mechanisms in vlms, 2025. 2, 6, 7
- [26] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. Feature visualization. *Distill*, 2017. <https://distill.pub/2017/feature-visualization>. 2
- [27] Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. *Transformer Circuits*, 2022. 2

- [28] Gonalo Paulo, Alex Mallen, Caden Juang, and Nora Belrose. Automatically interpreting millions of features in large language models, 2025. [8](#)
- [29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021. [1](#), [2](#)
- [30] Decode Research. SAE Dashboard. <https://github.com/jbloomAus/sae-dashboard>, 2024. [2](#), [6](#)
- [31] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115 (3):211–252, 2015. [5](#), [6](#)
- [32] Adam Scherlis, Kshitij Sachan, Adam S. Jermyn, Joe Benton, and Buck Shlegeris. Polysemanticity and capacity in neural networks, 2025. [2](#)
- [33] Seongbum Seo, Sangbong Yoo, Hyelim Lee, Yun Jang, Ji Hwan Park, and Jeong-Nam Kim. A sentence-level visualization of attention in large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*, pages 313–320, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. [1](#), [2](#)
- [34] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ram  , Morgane Rivi  re, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025. [1](#), [5](#), [6](#)
- [35] OpenAI Team. Gpt-4o system card, 2024. [1](#)
- [36] Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. [2](#)
- [37] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small. *arXiv preprint arXiv:2211.00593*, 2022. [2](#)
- [38] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *Computer Vision – ECCV 2014*, pages 818–833, Cham, 2014. Springer International Publishing. [2](#), [7](#)
- [39] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11975–11986, 2023. [5](#)
- [40] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. *arXiv preprint arXiv:2309.16042*, 2023. [2](#)